UNIVERSITÀ DI TORINO

Hybrid Workflows for Artificial Intelligence

Reproducibility in Computer Science

Prof. Iacopo Colonnelli

iacopo.colonnelli@unito.it

https://alpha.di.unito.it/iacopo-colonnelli

Why reproducibility?

We are facing a progressive **loss of confidence** in the global scientific community

Some reasons are:

- **Increasing distance** between edge scientific research (more an more complex) and broader audience
- Exponential increase in the **number of scientific publications**, making it difficult to keep up to date
- Increasing number of **retracted research works** due to flaws, haste, or frauds

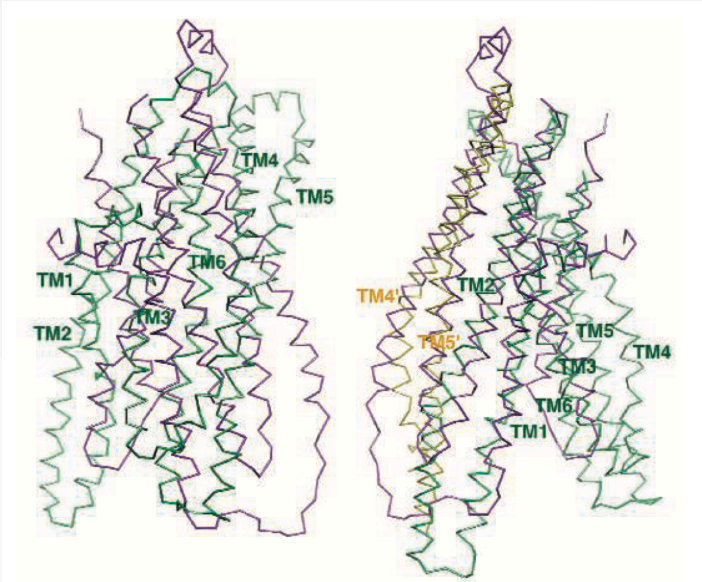
Example: Geoffrey Chang's articles

Around 2000 **Geoffrey Chang**, published a series of high-profile scientific works about the structures of multidrug resistance transporter proteins in bacteria

Thanks to them, he was awarded a **Presidential Early Career Award for Scientists and Engineers**, the USA highest honor for young researchers

In 2006, Dawson and Locher from ETH Zurich publish another article that raises **strong concerns** about Chang's work

A further investigation discovered a **data management error** made by Chang's group, which finally led to the retraction of five highly-cited articles



Flipping fiasco. The structures of MsbA (purple) and Sav1866 (green) overlap little (*left*) until MsbA is inverted (*right*).

Reproducibility in action

My supervisor asked me to reproduce the experiments described in a scientific paper. What should I do?

1. Download the article
2. Run the code on their data
3. Compare the results

Reproducibility in action

My supervisor asked me to reproduce the experiments described in a scientific paper. What should I do?

1. **Download the article**
   - Not all articles are accessible for free: many of them are **behind a paywall**
   - Sometimes articles come in **multiple versions**. Reproducibility details are often in appendices, which are not reported in the main version
2. Run the code on their data
3. Compare the results

Reproducibility in action

UNIVERSITÀ DI TORINO

My supervisor asked me to reproduce the experiments described in a scientific paper. What should I do?

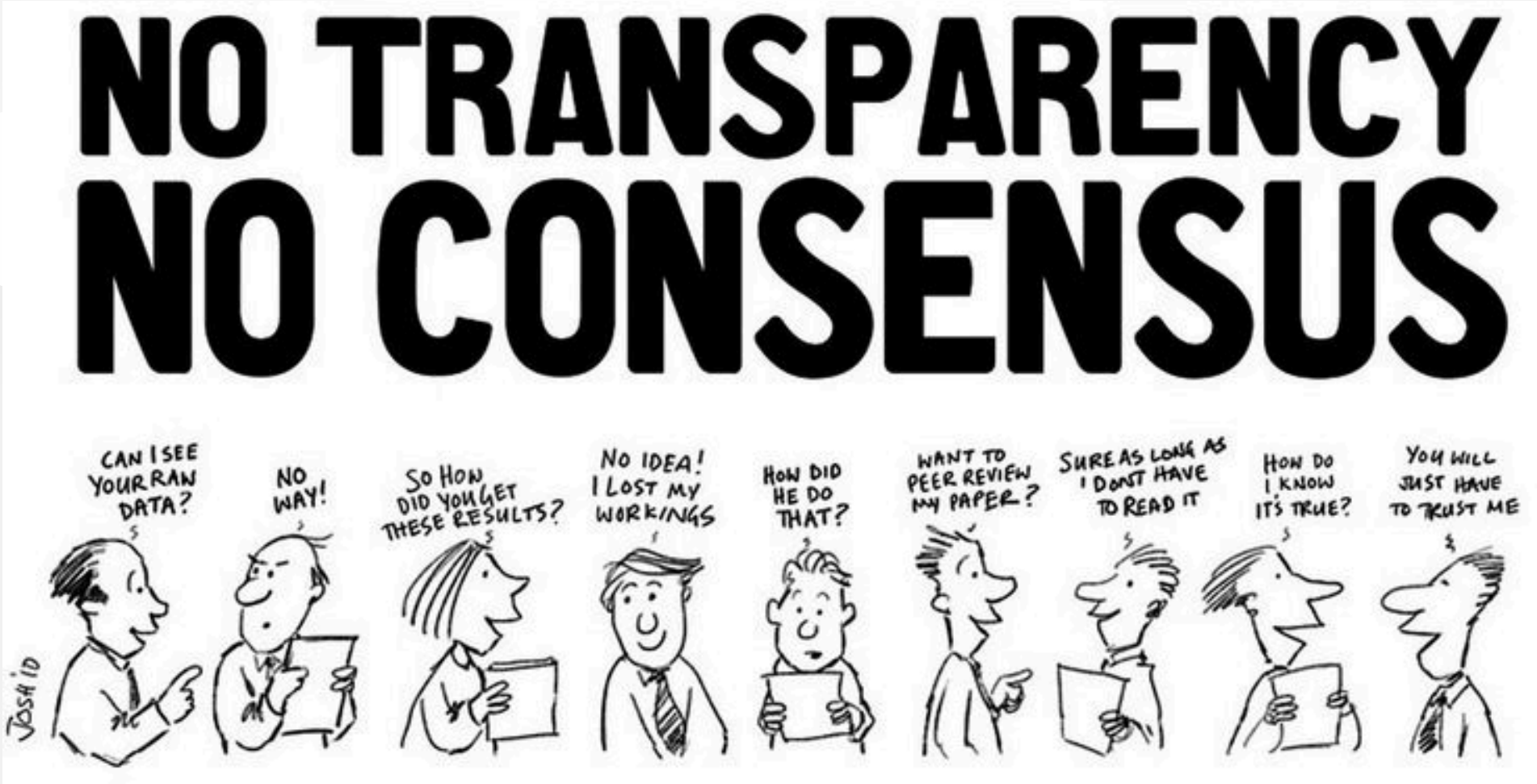
1. Download the article
2. **Run the code on their data**
   - Some articles do not contain (or link to) the code, but just describe the algorithms with **pseudocode**
   - Sometimes the article contains links to the code...but the **link is expired**!!!
   - For recent articles, the code is still there...but **dependencies are broken**!!!
   - Finding **open datasets** is even more difficult (heavy artifatcs, privacy concerns, ...)
3. Compare the results

Reproducibility in action

My supervisor asked me to reproduce the experiments described in a scientific paper. What should I do?

1. Download the article
2. Run the code on their data
3. **Compare the results**
   - Results differ: why?
     - Which **input parameters** have been used to run the executable?
     - How was the **execution environment** configured (hardware, operating system, compilers, external dependencies, ...)?
     - **How many times** and **in which order** did the author execute the experiment phases?
   - Results are comparable: good
     - Are they still valid on a **different dataset**?

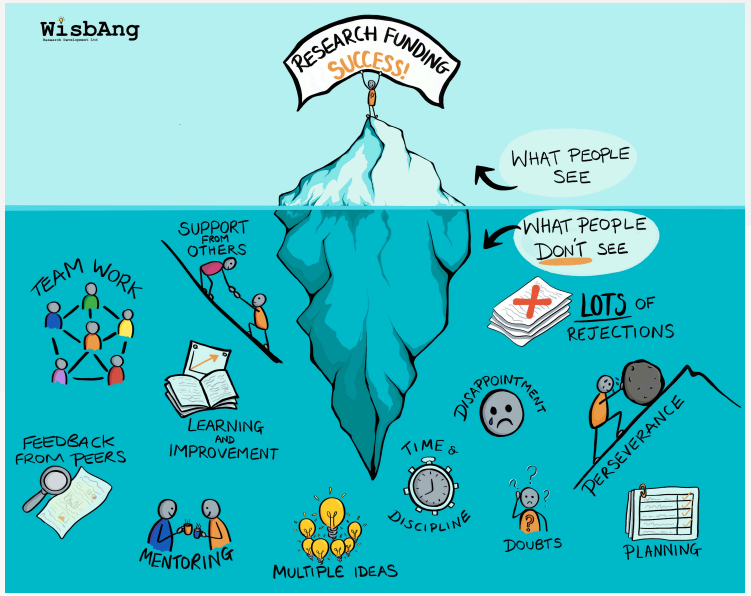
The curse of reproducibility



The curse of reproducibility

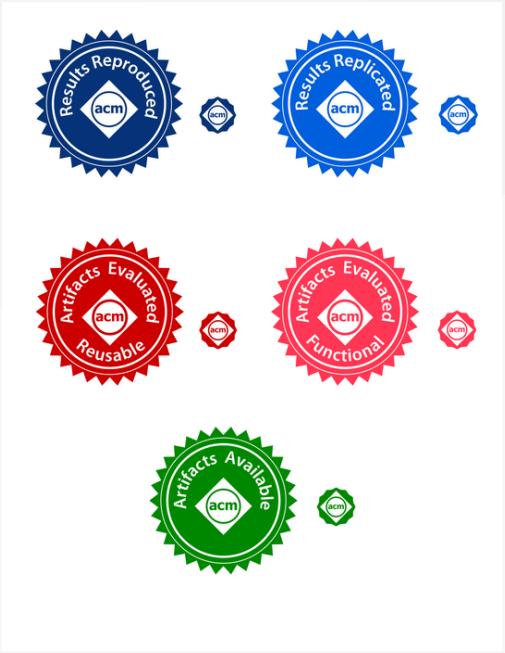
Scientific publications, even when valuable, are only the tip of the iceberg. Behind them, there are:

- **Incomplete documentation** of the experiment (due to limited space)
- Complex **execution workflows** (trial and error)
- Huge datasets, often preprocessed in a **semi-automatic** way
- A large amount of **computing hours**, with consequent high costs and power consumption



Reproducibility - Social challenges

- **Unwillingness to share** code and data (intellectual property, competition, ...)
- Perceived **inadequacy of the software** (dirty code, undocumented software, hacks, ...)
- **Bad peer review** (no time for reproducibility assesments, lack of detail in double-blind processes, ...)
- **Lack of incentives** (no rewards for reproducibility from journals, conferences, and academic institutions)



Reproducibility - Methodological challenges

- Use of **proprietary/commercial software** (license-related problems)
- Inadequacy of the **experimental plan** (no statistical significance, incorrect environment setup, ...)
- **Prejudices** (apophenia, selective results, ...)
- **Flawed data anaysis** (data preprocedding errors, computational errors, ...)
- **Poor documentation** (undescribed input parameters, no runtime specification, dependency locking partial or absent, ...)
- Improper usage of **statistics** (p-hacking, HARKing, ...)

Reproducibility in HPC - Challenges

Large-scale computing leverages on **parallel/distributed programming**, which introduces new challenges for reproducibility:

- **Out-of-order** floating point operations, due to the variability in the execution order of parallel instructions (thread scheduling, load balancing, ...)
- Lack of associative property due to machine **finite precision**, which is further enhanced when using **low-precision arithmetics**

In Mathematics:

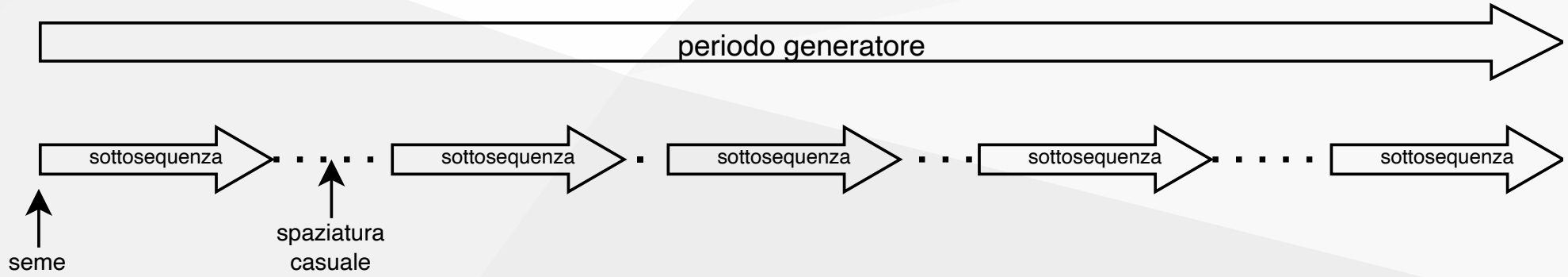
$$\begin{aligned} 10^{-3} &= 10^{-3} + 1 - 1 \\ &= 10^{-3} + (1 - 1) \\ &= 0.001 \end{aligned}$$

In Computer Science:

```
>>> pow(10, -3)
0.001
>>> pow(10, -3) + 1 - 1
0.0009999999999998899
>>> pow(10, -3) + (1 - 1)
0.001
```

Reproducibility in HPC - Challenges

- Scientific simulations often rely on **random number generators**, which commonly implement a **seed-dependent pseudorandom generation**
- In parallel simulations, the risk is to rely on the **same pseudorandom sequence** in wannabe-independent threads, making them (strongly) correlated
- Keeping track of **all the involved seeds** is a crucial step towards reproducibility



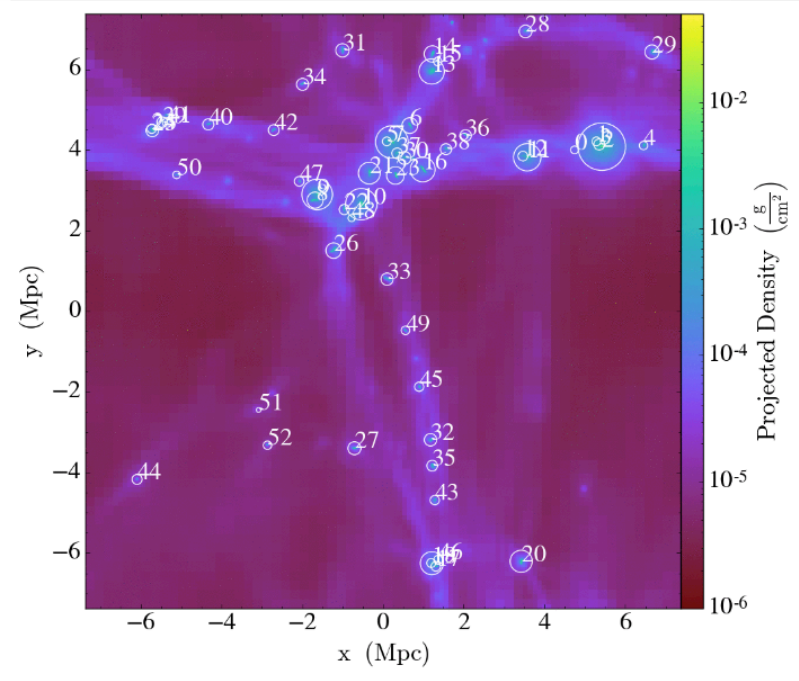
Reproducibility in HPC - Challenges

Performance **optimisation** is the crucial goal of HPC. However:

- Code compilers offer different **optimisation levels**, typically ranging from `O0` (do not optimise) to `O3` (apply the whole set of known optimisation paths with correctness guarantees), to `Ofast` (sacrifice math precision to improve performance)
- Compiler optimisations translates into **automatic code manipulation** (or, more often, one of its intermediate representations) to reduce the execution time (e.g., use advanced arithmetic instructions like FMA or vector operations)
- Different compilers (and different versions of the same compiler) may apply **different optimisations**

Reproducibility in HPC - Challenges

Compiler	Optimization	Walltime	Average [g]	Std Err [g]
gcc@6.2.0	None	22h	$2.2740 \cdot 10^{46}$	$1.07 \cdot 10^{44}$
	Normal	10h	$2.2668 \cdot 10^{46}$	$1.22 \cdot 10^{44}$
	High	9h	$2.2747 \cdot 10^{46}$	$1.20 \cdot 10^{44}$
intel@16.0.3	None	39h	$2.2715 \cdot 10^{46}$	$1.59 \cdot 10^{44}$
	Normal	7h	$4.3304 \cdot 10^{45}$	$1.25 \cdot 10^{44}$
	High	6h	$2.2685 \cdot 10^{46}$	$1.41 \cdot 10^{44}$
pgi@16.9.0	None	14h	$2.2701 \cdot 10^{46}$	$1.22 \cdot 10^{44}$
	Normal	13h	$2.2720 \cdot 10^{46}$	$1.33 \cdot 10^{44}$
	High	10h	$2.2713 \cdot 10^{46}$	$1.19 \cdot 10^{44}$
cce@8.5.5	None	16h	$4.3115 \cdot 10^{45}$	$1.35 \cdot 10^{44}$
	Normal	6h	$2.2713 \cdot 10^{46}$	$1.26 \cdot 10^{44}$
	High	5h	$2.2723 \cdot 10^{46}$	$1.34 \cdot 10^{44}$

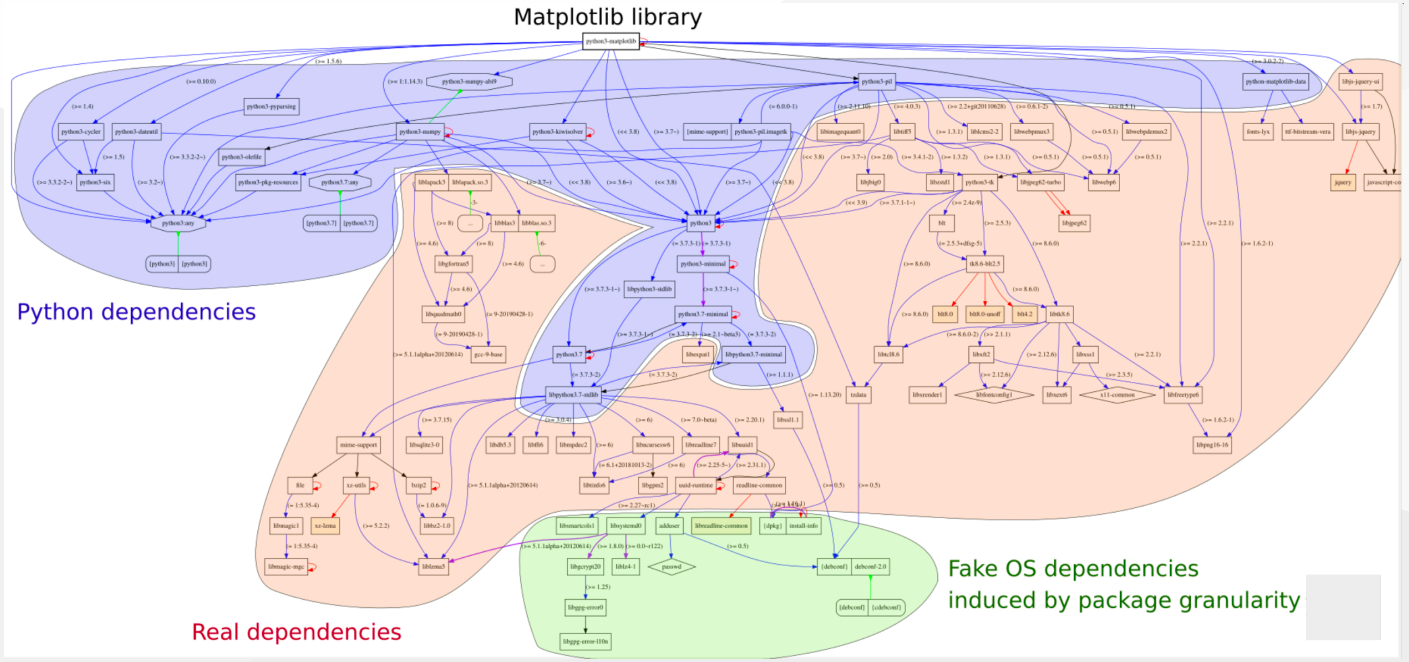


V. Stodden and M. S. Krafczyk. "Assessing reproducibility: An astrophysical example of computational uncertainty in the HPC context," in Proceedings of the 1st Workshop on Reproducible, Customizable and Portable Workflows for HPC at SC. Vol. 18. 2018.

Reproducibility in HPC - Challenges

Modern software libraries can hide **deep and complex depemndency graphs**. Failing to fix the version of even a small subset of indirect dependencies can lead to reproducibility issues

```
apt show python3-matplotlib
```



Reproducibility in HPC - Challenges

UNIVERSITÀ DI TORINO

HPC facilities are **heterogeneous systems** that differ significantly from each other

Workloads may vary significantly both through different systems, but also **within the same facility** (due to the dynamic resource assignment operated by the batch schedulers)

In addition, also rare events as **radiation-induced bit flips** become significant at the scale of modern data centers. Frontier (8.7M cores) has an estimated MTBF of few hours

	CINECA@Leonardo	CSC@Lumi
Nodes	3456 (Booster)	2978 (GPU part.)
CPU	32-core Intel Xeon 8358	64-core AMD Trento
RAM	512 (8x64) GB	256 - 1024 GB
GPU	4x Nvidia Ampere (64 GB)	4x AMD MI250X GPUs (128 GB)
Rete	2x 200 Gbit/s	4x 200 Gbit/s

Benjamin A. Antunes, David R. C. Hill. Reproducibility, Replicability, and Repeatability: A survey of reproducible research with a focus on high performance computing. CoRR abs/2402.07530 (2024).

UNIVERSITÀ DI TORINO

Use battle-tested linear algebra tools

Floating point arithmetic relies on **finite-precision numbers** due to the finite number of available bits to represent them: $a + b$ translates to $\text{round}(a + b)$

As a consequence, commonly $\text{round}(a + \text{round}(b + c)) \neq \text{round}(\text{round}(a + b) + c)$. However, advanced **linear algebra** techniques ensure that the error is negligible in most cases

Relying on production-tested, high-performance **linear algebra libraries** instead of implementing matrix operations from scratch helps limiting error propagation

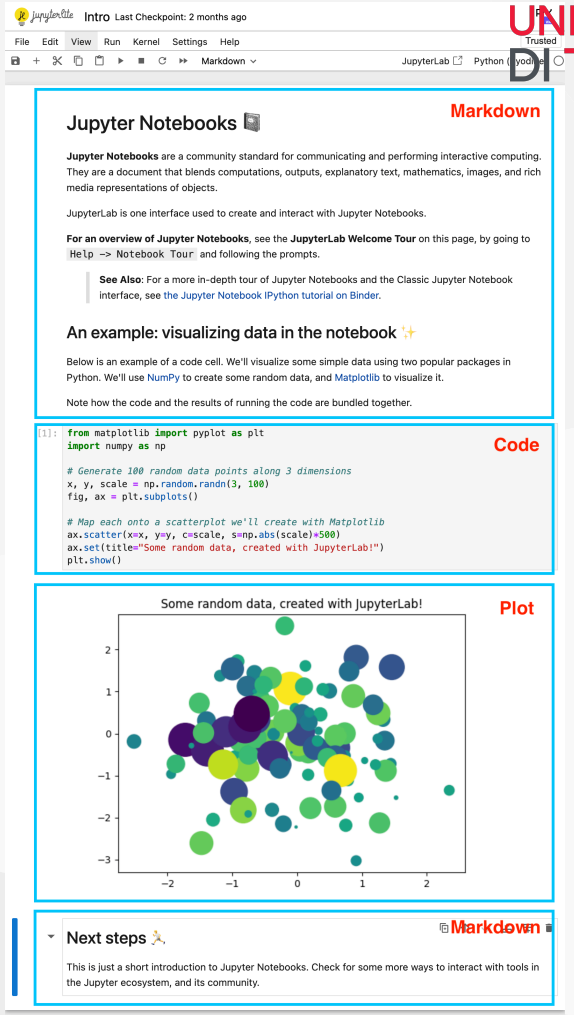
Document experiments

Reproducibility relies on a **careful documentation** of all the phases of a scientific experiment

It is crucial to keep track of the all the **code**, the **compiltion and execution parameters**, the **involved data** (input, output, and intermediate), potential **runtime interventions**, and resulting **observations**

The **literate computing** paradigm, introduced by Knuth in 1984, helps scientists in documenting their work by interleaving code and documentation in a unique, portable format

Nowadays, **Jupiter notebooks** are the most widesprerad tool for literate computing, supporting more than 40 languages

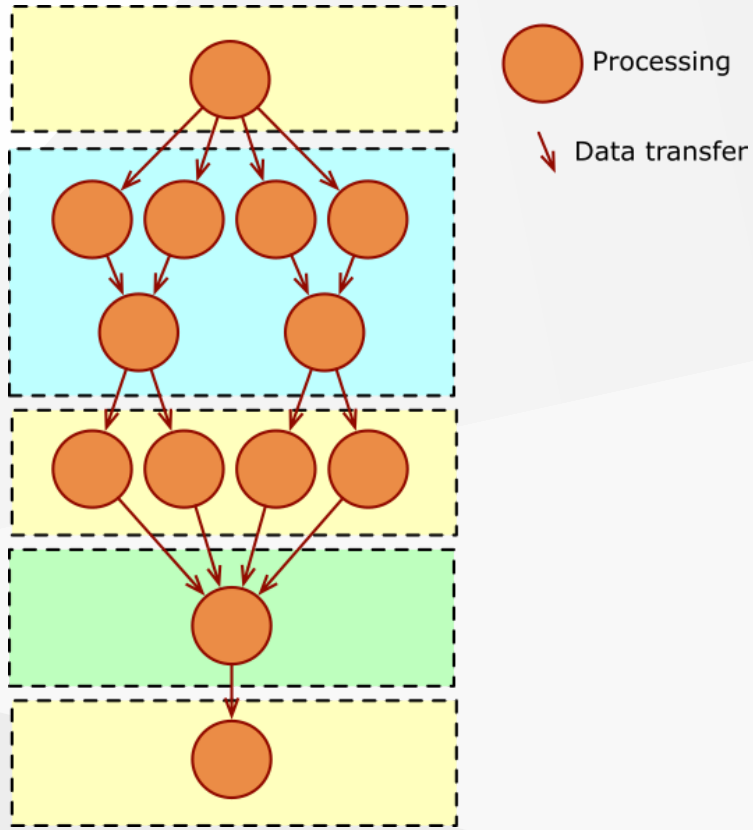


Write experiments as workflows

Reproducing a complex application may need users to perform **multiple tasks in the correct order**

Interative notebooks are prone to reproducibility issues, as they allow users to execute cells **in any order**, without documenting it

Conversely, designing a workflow forces users to explicitly formalise the **task inter-dependencies**, leading to reproducible executions



Rely on versioning tools

A common bad practice when rushing for a publication is to modify the code **directly in the experimental environment**

This practice hinders reproducibility, as it introduced the risk that the **published version** of the code differs from the one actually used to collect observations

Relying on **versioning tools** to document and keep track of all code updates is crucial for long-term reproducibility

A **Version Control System (VCS)** (Git, SVN, ...) also allows users to **compare** different versions, **restore** a previous snapshot, manage **multiple versions** in parallel, incrementally **update** and automatically **test** the code (e.g., updating dependency versions), and **share** the code with other developers

Rely on provenance tools

VCSs are powerful tools, but they still require **manual checks** to ensure that the code stored in the VCS coincides with the one that generated the outputs

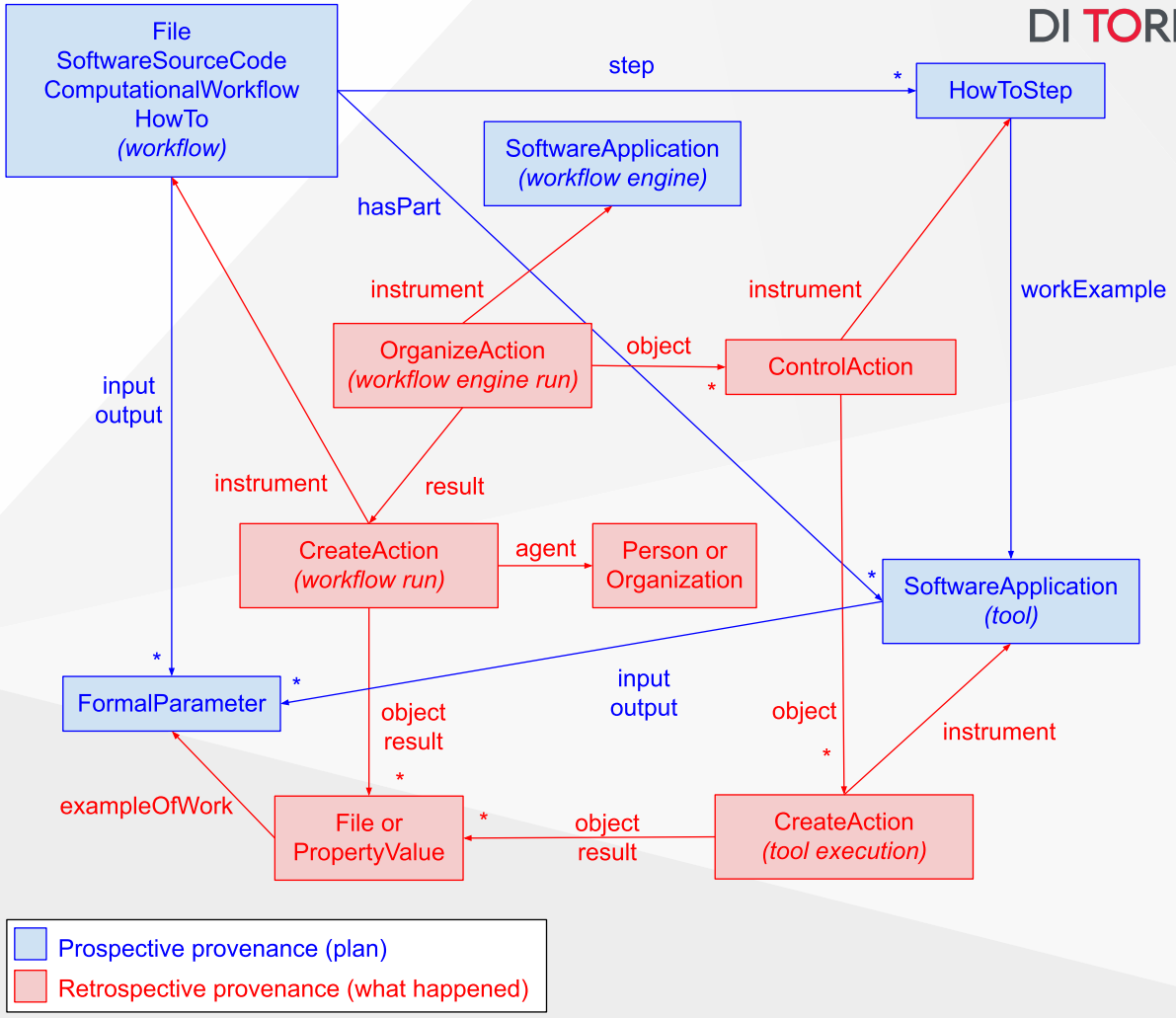
On the other hand, **provenance tools** complement workflow executions with metadata that document each workflow step (**prospective provenance**) and its whole execution (**retrospective provenance**)

In particular, provenance formats usually provide **checksums** for each code and data object involved in an experiment (including intermediate data). Companion tools then implement **automated verification** of provenance archives, checking for integrity and correctness

Example: the Workflow Run RO-Crate profile

The **Workflow Run RO-Crate** profile collection extends **RO-Crate** (Research Object Crate) and **Schema.org** to capture the provenance of computational workflows at different levels of granularity and bundle together their associated products (inputs, outputs, code, …)

The **Provenance Run Crate** profile, the most extended schema in the collection, describes internal details of the workflow run, such as step executions and intermediate outputs



UNIVERSITÀ DI TORINO

Document the software environment

Recreating the execution environment of a modern scientific experiment is a complex task, especially due to the heterogeneity of HPC facilities

Modern languages (Java, Python, JavaScript, Rust, ...) provide **official package managers**, i.e., repositories where developers can publish and version software libraries (Maven/Gradle, PyPI/Conda, NPM, Crares, ...)

However, legacy languages commonly used in HPC applications (C, C++, Fortran, ...) do not provide a package manager out of the box

Environment Modules

The **Modules package** is simplifies shell initialization and lets users easily modify their environment during a session

Each **modulefile** contains the information needed to configure the shell for an application. Typically, modulefiles alter or set shell environment variables (`PATH` , `MANPATH` , ...)

Modules can be loaded and unloaded **dynamically** and **atomically**, in an clean fashion. All popular shells are supported

```
module load openmpi
```

The Spack package manager

Spack is a package management tool designed to support multiple versions and configurations of software on a **wide variety of platforms and environments**

Spack was designed for **large supercomputing centers**, where many users and application teams share common installations of software on clusters with exotic architectures, using libraries that do not have a standard ABI

Spack is **non-destructive**: installing a new version does not break existing installations, so many configurations can coexist on the same system.

Spack

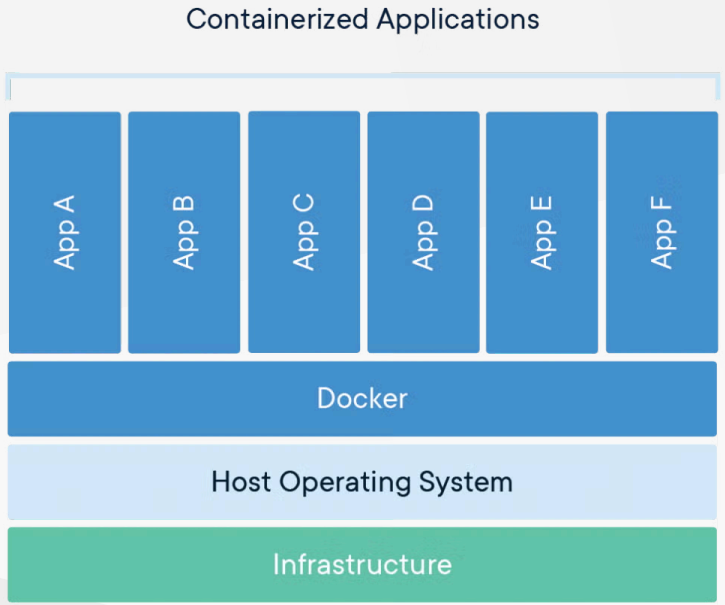
```
spack install openmpi
```

Software containers

Software containers are **packages of software** that contain all of the necessary elements to run in any environment

A containers **virtualizes the operating system** and run anywhere, from a private data center to the public cloud or even on a developer's personal laptop

However, software containers **rely on the host OS kernel**, strongly reducing the overhead brought by traditional VMs



Apptainer

Traditional container runtimes (Docker, containerd, ...) require **root privileges** to build and run containers, mainly to deal with network isolation

Apptainer (formerly Singularity) was created to run complex applications on HPC clusters in a simple, portable, and reproducible way without requiring root privileges

Despite implementing a different software stack, Apptainer is **compatible with Docker images**. Apptainer users can directly download (and transparently convert) Docker images, and Apptainer images can seamlessly extend Docker images

Reproducibility-oriented OSes

Few modern Linux distribution have been designed and developed to ensure **reproducibility of the full OS** environment

NixOS relies on the Nix package manager, which implements:

- Immutable ecosystem with **atomic updates**
- **Declarative configuration system** for easily sharing of the entire software environment

Guix is a package manager for GNU/Linux systems which:

- Supports **transactional** updates and rollbacks
- Allows for a **declarative description** of the whole execution environment

Archive and share scientific results

UNIVERSITÀ DI TORINO

All digital assets coming from an experiment should follow the **FAIR data principles**, established by Wilkinson et al. in 2016:

- **Findability**
  - (Meta)data should be assigned a **globally unique and persistent identifier**
  - Data should be described with **rich metadata**
  - Metadata should clearly and explicitly **include the identifier** of the data it describes
  - (Meta)data should be registered or indexed in a **searchable resource**
- **Accessibility**
  - (Meta)data should be retrievable by their identifier using a **standardized communications protocol**. The protocol should be **open**, **free**, and **universally implementable**, and it should allow for an **authentication** and **authorization** procedure, where necessary
  - Metadata should be **accessible**, even when the data are no longer available

Archive and share scientific results

All digital assets coming from an experiment should follow the **FAIR data principles**, established by Wilkinson et al. in 2016:

- **Interoperability**:
  - (Meta)data should use a formal, accessible, shared, and broadly applicable **language for knowledge representation**
  - (Meta)data should use **vocabularies** that follow FAIR principles
  - (Meta)data should include **qualified references** to other (meta)data
- **Reusability**
  - Meta(data) should be richly described with a plurality of **accurate and relevant attributes**. They should be released
  - (Meta)data should be released with a clear and accessible **data usage license**
  - (Meta)data should be associated with **detailed provenance**
  - (Meta)data should meet domain-relevant **community standards**

Archive and share scientific results

FAIR principles are quite generic, but efforts have been made to map them onto more concrete domains, like **research software** and **computational workflows**

Relying on **open standards** for archiving experimental data helps interoperability and reusability across a broad community of adopters

Archiving experimental results on **public registries** improves findability and accessibility

Workflow languages systems normally implement **out-of-the-box compatibility** with the most widespread archival standard and artifact registries, usually privileging open solutions to vendor-specific alternatives